

Fundamentos de Desenvolvimento de Software

Relatório Técnico do Projeto

Aplicação de Gestão de Escola de Boxe

Alunos: Leandro Pinheiro (1241358), Rui Menezes (1241394), Miguel Sousa (1241369), Henrique Valente (1241391)

Curso: LETI
Unidade curricular: FSOF
Ano letivo: 2025/2026
Data: 15 de junho de 2026

Instituto Superior de Engenharia do Porto
Documentação do projeto e dos padrões utilizados

Índice

1. Introdução
2. Objetivos da aplicação
3. Estrutura e organização do projeto
4. Funcionalidades implementadas
5. Arquitetura da solução
6. Modelo de domínio
7. Padrões e princípios utilizados
8. Tratamento das relações por referência
9. Testes e ProjectTester
10. Conclusão
11. Referências

1. Introdução

O presente relatório documenta o desenvolvimento de uma aplicação em C++ para gestão de uma escola de boxe. A aplicação permite gerir alunos, planos de treino e matrículas, criando uma ligação entre cada aluno e os planos de treino em que se encontra inscrito.

A documentação foi estruturada de forma técnica, dando destaque à organização do projeto, às camadas utilizadas, às responsabilidades de cada classe e aos padrões/princípios de desenho aplicados. Foram incluídos excertos curtos de código apenas quando estes servem para comprovar a aplicação de um padrão ou decisão arquitetural.

Nota: os excertos de código apresentados são parciais e servem apenas como evidência das decisões de desenho. O código completo pertence ao projeto em C++.

2. Objetivos da aplicação

A aplicação tem como objetivo principal organizar a informação de uma escola de boxe de forma simples, modular e orientada a objetos.

- Registrar, consultar, atualizar, listar e remover alunos.
- Registrar, consultar, atualizar, listar e remover planos de treino.
- Registrar matrículas de alunos em planos de treino.
- Consultar os planos de treino associados a um aluno.
- Consultar os alunos associados a um plano de treino.
- Evitar inconsistências, como remover um aluno ou plano de treino ainda associado a matrículas.
- Separar as responsabilidades em camadas: View, Controller, Service, Model, Mapper, DTO, Repository e Exceptions.

3. Estrutura e organização do projeto

O projeto encontra-se organizado por responsabilidades. Esta divisão facilita a leitura, a manutenção e a evolução da aplicação, evitando concentrar toda a lógica numa única classe.

Pasta / Camada	Responsabilidade principal	Exemplos
controller	Coordena os menus e chama os serviços adequados.	Controller
view	Interação com o utilizador, leitura de dados e impressão de resultados.	View, StudentView, TrainingPlanView, MatriculaView
service	Contém a lógica de negócio da aplicação.	StudentService, TrainingPlanService, MatriculaService
model	Representa as entidades principais e os containers.	Person, Student, TrainingPlan, Matricula, School, Exercise
mappers	Converte objetos do modelo em DTOs.	StudentMapper, TrainingPlanMapper, MatriculaMapper
dto	Transporta dados entre camadas sem expor diretamente o modelo.	StudentINDTO, StudentOUTDTO, MatriculaOUTDTO
repository	Abstrai o acesso ao modelo de dados.	ISchoolRepository, SchoolMemoryRepository
exceptions	Centraliza exceções da aplicação.	InvalidDataException, NotFoundException, DataConsistencyException

3.1 Visão geral da arquitetura

A figura seguinte resume a comunicação entre as camadas principais da aplicação.

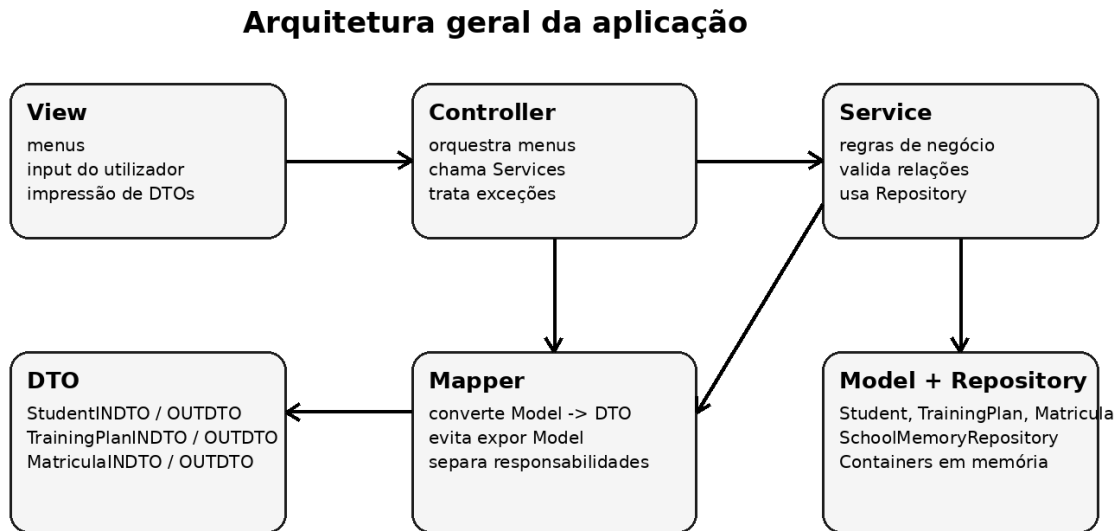


Figura 1 - Arquitetura geral da aplicação.

4. Funcionalidades implementadas

Módulo	Funcionalidades
Alunos	Adicionar aluno, consultar aluno, remover aluno, atualizar aluno e listar alunos.
Planos de treino	Adicionar plano de treino, consultar plano, remover plano, atualizar plano, listar planos e adicionar exercícios.
Matrículas	Registrar matrícula, consultar matrículas de um aluno, remover matrícula, listar matrículas e consultar alunos inscritos num plano de treino.

4.1 Fluxo de utilização

O utilizador interage com a aplicação através de menus. O menu principal permite escolher entre alunos, planos de treino e matrículas. Cada submenu apresenta opções específicas e recolhe os dados necessários através das Views.

```
int View::menuSchool() {  
    cout << endl;  
    cout << "===== BOXING SCHOOL =====" << endl;  
    cout << "1 - Students" << endl;  
    cout << "2 - Training Plans" << endl;  
    cout << "3 - Matriculas" << endl;  
    cout << "0 - Exit" << endl;  
  
    return Utils::getNumber("Option");  
}
```

5. Arquitetura da solução

5.1 View

A camada View é responsável pela comunicação com o utilizador. Esta camada lê dados introduzidos no terminal e apresenta resultados. Por exemplo, a `MatriculaView` recolhe os identificadores necessários para criar uma matrícula e constrói um DTO de entrada.

```
MatriculaINDTO MatriculaView::getMatricula() {
    int studentId = Utils::getNumber("Enter the Student Id");
    int trainingPlanId = Utils::getNumber("Enter the Training Plan Id");

    int day = Utils::getNumber("Enter the Day");
    int month = Utils::getNumber("Enter the Month");
    int year = Utils::getNumber("Enter the Year");

    DateDTO date(day, month, year);
    MatriculaINDTO matricula(studentId, trainingPlanId, date);

    return matricula;
}
```

5.2 Controller

O Controller recebe as opções escolhidas pelo utilizador e encaminha o pedido para o serviço correto. Também captura exceções da aplicação e envia mensagens para a View.

```
Controller::Controller(
    StudentService *studentService,
    TrainingPlanService *trainingPlanService,
    MatriculaService *matriculaService
) {
    this->studentService = studentService;
    this->trainingPlanService = trainingPlanService;
    this->matriculaService = matriculaService;
}
```

5.3 Service

A camada Service contém a lógica de negócio. É aqui que os IDs vindos da View/DTO são transformados em referências para os objetos reais do modelo.

```
Student *student = studentContainer.get(obj.studentId);
TrainingPlan *trainingPlan = trainingPlanContainer.get(obj.trainingPlanId);

matriculaContainer.add(
    id,
    student,
    trainingPlan,
    obj.date.day,
    obj.date.month,
    obj.date.year
);
```

5.4 Model

O Model representa as entidades principais da aplicação. As classes mais importantes são Person, Student, TrainingPlan e Matricula. A classe School agrega os containers principais e funciona como ponto de acesso ao modelo em memória.

```
StudentContainer& School::getStudentContainer() {  
    return this->studentContainer;  
}  
  
TrainingPlanContainer& School::getTrainingPlanContainer() {  
    return this->trainingPlanContainer;  
}  
  
MatriculaContainer& School::getMatriculaContainer() {  
    return this->matriculaContainer;  
}
```

6. Modelo de domínio

Classe	Descrição	Responsabilidade
Person	Classe base com id, nome, telefone e email.	Validar e armazenar dados comuns de uma pessoa.
Student	Representa um aluno da escola de boxe.	Especializa Person para o domínio dos alunos.
TrainingPlan	Representa um plano de treino.	Guardar nome, objetivo, duração em semanas e exercicios.
Matricula	Representa a inscrição de um aluno num plano de treino.	Ligar um Student a um TrainingPlan e guardar a data da matrícula.
School	Agrega os containers da aplicação.	Disponibilizar acesso aos dados em memória.

6.1 Classe Person e validação de dados

A classe Person contém validações simples para garantir que os dados base são coerentes. Isto evita que objetos inválidos entrem no sistema.

```
void Person::setEmail(const string& email) {  
    if (!isEmailValid(email)) {  
        throw InvalidDataException("Invalid email.");  
    }  
  
    this->email = email;  
}
```

6.2 Classe TrainingPlan

A classe TrainingPlan representa um plano de treino com nome, objetivo e duração. A duração deve ser positiva, garantindo uma regra simples de consistência.

```
void TrainingPlan::setDurationWeeks(int durationWeeks) {
    if (!isDurationWeeksValid(durationWeeks)) {
        throw InvalidDataException("Invalid training plan duration.");
    }

    this->durationWeeks = durationWeeks;
}
```

6.3 Classe Matricula

A classe Matricula é uma das classes centrais, porque representa a relação entre um aluno e um plano de treino. Em vez de guardar apenas o id do aluno e do plano, a matrícula guarda referências diretas para os objetos Student e TrainingPlan.

```
Matricula::Matricula(
    int id,
    Student *student,
    TrainingPlan *trainingPlan,
    int day,
    int month,
    int year
) : id(id), student(student), trainingPlan(trainingPlan) {

    if (student == nullptr) {
        throw invalid_argument("Invalid student.");
    }

    if (trainingPlan == nullptr) {
        throw invalid_argument("Invalid training plan.");
    }
}
```

7. Padrões e princípios utilizados

Nesta secção são apresentados os principais padrões e princípios utilizados no projeto. Cada padrão é acompanhado de uma breve explicação e de um excerto curto de código que comprova a sua aplicação.

7.1 SRP - Single Responsibility Principle

O princípio SRP indica que cada classe deve ter uma única responsabilidade. No projeto, esta ideia aparece na divisão entre Services, Views, Models, Mappers, DTOs e Exceptions.

Classe	Responsabilidade única
StudentService	Lógica de negócio associada a alunos.
TrainingPlanService	Lógica de negócio associada a planos de treino.
MatriculaService	Lógica de negócio associada a matrículas.
StudentMapper	Conversão de Student para StudentOUTDTO.
View / StudentView / MatriculaView	Interação com o utilizador.

```
void StudentMapper::model2DTO(Student* student, StudentOUTDTO& dto) {
    dto.id = student->getId();
    dto.name = student->getName();
    dto.phoneNumber = student->getPhoneNumber();
    dto.email = student->getEmail();
}
```

Neste exemplo, o StudentMapper não valida alunos, não lê input e não guarda dados. A sua responsabilidade é apenas converter um objeto Student num DTO.

7.2 DTO - Data Transfer Object

O padrão DTO é usado para transportar dados entre camadas sem expor diretamente os objetos do domínio. Esta abordagem ajuda a proteger o modelo e reduz o acoplamento entre a View e o Model.

DTO	Utilização
StudentINDTO	Dados introduzidos para criar/atualizar um aluno.
StudentOUTDTO	Dados apresentados ao consultar/listar alunos.
TrainingPlanINDTO	Dados introduzidos para criar/atualizar um plano de treino.
TrainingPlanOUTDTO	Dados apresentados ao consultar/listar planos.
MatriculaINDTO	Dados necessários para registar uma matrícula.
MatriculaOUTDTO	Dados completos de uma matrícula para apresentação.

```
StudentINDTO student = this->studentView.getStudent();
this->studentService->add(student);
```

O Controller recebe um StudentINDTO vindo da View e envia-o para o Service, mantendo a classe Student escondida da interface.

7.3 Mapper

O padrão Mapper é utilizado para transformar objetos do modelo em DTOs. Esta separação evita que o Model conheça a forma como os dados são apresentados e evita que a View dependa da estrutura interna das entidades.

```
Student *student = matricula->getStudent();
TrainingPlan *trainingPlan = matricula->getTrainingPlan();

dto.studentId = student->getId();
dto.studentName = student->getName();
dto.trainingPlanId = trainingPlan->getId();
dto.trainingPlanName = trainingPlan->getName();
```

Neste excerto, o MatriculaMapper extrai informação da Matricula, do Student e do TrainingPlan, criando um DTO pronto para ser apresentado pela View.

7.4 Repository Pattern

O Repository Pattern é usado para separar a lógica de acesso aos dados da lógica de negócio. No projeto existe uma interface de repositório e uma implementação em memória, o que permite substituir no futuro a persistência em memória por persistência em ficheiros.


```

SchoolMemoryRepository::SchoolMemoryRepository() {
    this->school = new School();
}

SchoolMemoryRepository::~SchoolMemoryRepository() {
    delete this->school;
}

School* SchoolMemoryRepository::getModel() {
    return this->school;
}

```

A implementação atual usa `SchoolMemoryRepository`. Esta abordagem está alinhada com a ideia de `Repository`, pois os `Services` dependem de uma abstração para obter o modelo, em vez de criarem diretamente os dados.

7.5 Dependency Injection

A injeção de dependências reduz o acoplamento entre classes. O `Controller` recebe os serviços no construtor e os `Services` recebem um `ISchoolRepository`. Assim, as dependências são fornecidas a partir de fora.

```

MatriculaService::MatriculaService(ISchoolRepository *repo) {
    this->repo = repo;
}

```

```

Controller::Controller(
    StudentService *studentService,
    TrainingPlanService *trainingPlanService,
    MatriculaService *matriculaService
) {
    this->studentService = studentService;
    this->trainingPlanService = trainingPlanService;
    this->matriculaService = matriculaService;
}

```

7.6 OCP - Open/Closed Principle

O princípio OCP indica que o software deve estar aberto a extensão, mas fechado a modificação. No projeto, este princípio é parcialmente suportado pela utilização de uma interface de repositório. Caso se pretenda guardar dados em ficheiros no futuro, pode ser criada uma nova classe, por exemplo `SchoolFileRepository`, sem alterar a lógica principal dos `Services`.

```

// Exemplo de evolução possível
// ISchoolRepository
//   |- SchoolMemoryRepository
//   |- SchoolFileRepository

```

O projeto atual usa repositório em memória. A persistência em ficheiro pode ser adicionada como uma extensão futura, mantendo a mesma interface.

7.7 Padrões não utilizados diretamente

Os padrões Singleton e Observer foram estudados nas aulas teóricas, mas não foram usados diretamente nesta versão do projeto. Não foi criado um Singleton porque a aplicação já recebe as dependências explicitamente. Também não foi criado um Observer porque não existe requisito de notificação automática quando uma entidade muda de estado.

8. Tratamento das relações por referência

Uma melhoria importante do projeto foi garantir que as relações internas do domínio são feitas por referência/ponteiro para objetos, e não apenas por IDs. Os IDs continuam a ser usados na View e nos DTOs, porque o utilizador identifica alunos e planos pelo número. Contudo, antes de chegar ao Model/Container, o Service converte esses IDs nos objetos correspondentes.

Camada	Utilização de IDs	Utilização de referências
View	Pode pedir IDs ao utilizador.	Não precisa de trabalhar com ponteiros.
DTO	Pode transportar IDs introduzidos pelo utilizador.	Não representa relações internas.
Service	Recebe IDs e procura os objetos reais.	Passa Student* e TrainingPlan* para o Model/Container.
Model / Container	Evita usar IDs para representar relações entre entidades.	Trabalha com Student* e TrainingPlan*.

8.1 Antes da correção

```
// Exemplo antigo, menos correto
matriculaContainer.getByStudent(studentId);
matriculaContainer.getByTrainingPlan(trainingPlanId);
```

8.2 Depois da correção

```
Student *student = studentContainer.get(studentId);
TrainingPlan *trainingPlan = trainingPlanContainer.get(trainingPlanId);

list<Matricula*> matriculasStudent = matriculaContainer.getByStudent(student);
list<Matricula*> matriculasPlan = matriculaContainer.getByTrainingPlan(trainingPlan);
```

Desta forma, a relação entre Matricula, Student e TrainingPlan fica coerente com o modelo orientado a objetos. A Matricula conhece os objetos associados, e não apenas números soltos.

8.3 Consistência e exceções

O projeto usa exceções para impedir estados inválidos ou operações inconsistentes, como remover um aluno que ainda tem matrículas associadas.

```
list<Matricula*> matriculas = matriculaContainer.getByStudent(student);

if (matriculas.empty()) {
    student = studentContainer.remove(id);
    StudentMapper::model2DTO(student, dto);
    delete student;
} else {
    string msg = "Student: " + to_string(id);
    throw DataConsistencyException(msg);
}
```

10. Testes e ProjectTester

Segundo a indicação dada na aula, a solução deve estar separada entre o projeto principal e um projeto de testes. O ProjectTester deve incluir o código do projeto principal como biblioteca ou dependência e executar funções de teste.

10.1 Estrutura esperada

```
Project/
  headers/
  Sources/
  CMakeLists.txt

ProjectTester/
  tests/
  main.cpp
  CMakeLists.txt
  // inclui o código/biblioteca do Project
```

10.2 Exemplo de teste

Os testes devem verificar funcionalidades essenciais, como adicionar, consultar e remover dados. O exemplo seguinte mostra uma função simples de teste para StudentContainer.

```
void testStudentContainer() {
    StudentContainer container;

    container.add(1, "Joao Silva", "912345678", "joao@email.com");

    Student* student = container.get(1);

    if (student->getName() != "Joao Silva") {
        throw runtime_error("Student test failed");
    }
}
```

10.3 Chamada da função test()

No projeto de testes, o main deve chamar uma função test() ou funções equivalentes, garantindo que os testes são executados separadamente do programa principal.

```
void test() {
    testStudentContainer();
    // testTrainingPlanContainer();
    // testMatriculaContainer();
}

int main() {
    test();
    return 0;
}
```

12. Conclusão

O projeto desenvolvido apresenta uma aplicação modular para gestão de uma escola de boxe. A solução permite gerir alunos, planos de treino e matrículas, mantendo uma separação clara entre apresentação, controlo, lógica de negócio e modelo de domínio.

Foram aplicados princípios e padrões importantes, como SRP, DTO, Mapper, Repository Pattern e Dependency Injection. A relação entre alunos e planos de treino foi também tratada de forma orientada a objetos, através de referências/ponteiros para Student e TrainingPlan dentro da Matricula.

A documentação evidencia que a arquitetura está preparada para evolução futura, nomeadamente a introdução de persistência em ficheiros e a criação de testes automatizados através do ProjectTester.

13. Referências

- Slides FSOF - T10: C++ File Handling and Repository Design Pattern.
- Slides FSOF - T11: Design Pattern (+), incluindo DTO e Mapper.
- Código-fonte do projeto: Controller, Views, Services, Models, Mappers, Repository e Exceptions.